



Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências – Bauru
Departamento de Computação

Título do Software

Subtítulo do Software

Engenharia de Software II

Nome do Autor 1 – RA: XXXXXXXX

Nome do Autor 2 – RA: XXXXXXXX

Nome do Autor 3 – RA: XXXXXXXX

Nome do Autor 4 – RA: XXXXXXXX

Sumário

Como utilizar este modelo	1
1 Introdução e Objetivos	2
1.1 Contexto do software	2
1.2 Escopo	2
1.3 Visão geral dos requisitos	2
1.4 Objetivos de qualidade	2
2 Arquitetura do Sistema	4
2.1 Contexto e fronteiras do sistema	4
2.2 Estilo ou padrão arquitetural	4
2.3 Contêineres e componentes	4
2.4 Decisões arquiteturais relevantes	4
2.5 Restrições de arquitetura	5
3 Projeto de Componentes	6
3.1 Visão geral dos componentes	6
3.2 Detalhamento dos componentes principais	6
3.3 Princípios de projeto	6
3.4 Padrões de projeto	7
4 Projeto de Interface do Usuário	8
5 Especificação e Estratégia de Testes	9
5.1 Planejamento e níveis de teste	9
5.2 Testes de unidade	9
5.3 Testes de integração	9
5.4 Critérios de conclusão e cobertura	9
5.5 Automação e ferramentas	9
6 Gestão de Configuração e Manutenção	10
6.1 Repositório e itens de configuração	10
6.2 Gestão de dependências e alterações	10
6.3 Rastreabilidade	10
6.4 <i>Build</i> , integração e entrega	10
7 Controle de Versões do Documento	11

Como utilizar este modelo

Este documento é um roteiro para apoiar a documentação do projeto final. Os textos marcados como *Orientação* explicam o que se espera em cada parte e podem ser ocultados alterando-se, no início do arquivo, `\orientacoestrue` para `\orientacoesfalse`.

O documento deve registrar as principais decisões do projeto, e não apenas reunir diagramas. Sempre que uma escolha relevante for apresentada, expliquem brevemente a motivação e suas consequências. Diagramas, tabelas e protótipos devem ser mencionados e explicados no texto.

Não é necessário documentar todas as classes, telas ou detalhes internos do sistema. Em projetos maiores, priorizem os elementos mais relevantes, complexos ou representativos.

Capítulo 1

Introdução e Objetivos

Orientação: Apresentem brevemente o sistema e expliquem o que será documentado neste capítulo. O objetivo é permitir que um leitor compreenda o problema e o escopo antes de examinar as decisões de projeto.

1.1 Contexto do software

Orientação: Contextualizem o problema do ponto de vista dos usuários e da atividade que o software deve apoiar, automatizar ou melhorar. Identifiquem os principais usuários ou partes interessadas quando isso for relevante.

1.2 Escopo

Orientação: Delimitem o que faz parte do sistema e, quando necessário, o que está explicitamente fora do escopo. Evitem transformar esta seção em uma especificação detalhada de requisitos.

1.3 Visão geral dos requisitos

Orientação: Apresentem os requisitos funcionais e não funcionais mais importantes para compreender o projeto. Caso a equipe tenha uma especificação detalhada de requisitos produzida anteriormente, ela pode ser incluída em um apêndice ou referenciada neste documento.

1.4 Objetivos de qualidade

Orientação: Identifiquem os atributos de qualidade que realmente influenciam as decisões de projeto, por exemplo: manutenibilidade, desempenho, segurança, confiabilidade, usabilidade ou portabilidade. Podem ser utilizados modelos como ISO/IEC 25010 ou FURPS+, desde que apenas os atributos relevantes ao sistema sejam selecionados.

Uma forma de sintetizar os objetivos e suas implicações é:

Tabela 1.1: Exemplo de objetivos de qualidade.

Objetivo	Prioridade	Impacto esperado no projeto
Manutenibilidade	Alta	Ex.: modularização, baixo acoplamento e testes automatizados.
Segurança	Alta	Ex.: autenticação, autorização e proteção de dados.
Usabilidade	Média	Ex.: consistência da interface e redução de etapas em tarefas frequentes.

Capítulo 2

Arquitetura do Sistema

Orientação: Apresentem a arquitetura proposta e justifiquem as principais decisões. O leitor deve conseguir compreender a estrutura geral do sistema antes de entrar nos detalhes de seus componentes.

2.1 Contexto e fronteiras do sistema

Orientação: Apresentem um diagrama de contexto mostrando o sistema, seus principais usuários e os sistemas externos com os quais ele se comunica. Expliquem as relações mostradas no diagrama.

2.2 Estilo ou padrão arquitetural

Orientação: Identifiquem o(s) estilo(s) ou padrão(ões) arquitetural(is) adotado(s), quando aplicável, e justifiquem a escolha considerando as características do sistema e seus objetivos de qualidade.

2.3 Contêineres e componentes

Orientação: Apresentem uma visão de alto nível da decomposição do sistema. Pode-se utilizar, quando adequado, os níveis de contexto, contêiner e componente do modelo C4. Não basta inserir os diagramas: descrevam a responsabilidade dos elementos relevantes e as principais dependências ou comunicações entre eles.

2.4 Decisões arquiteturais relevantes

Orientação: Registrem as decisões que tiveram impacto significativo na arquitetura. Para cada decisão, apresentem de forma breve a motivação, alternativas consideradas quando pertinente e principais consequências.

Tabela 2.1: Modelo para registrar decisões arquiteturais.

Decisão	Motivação	Consequências
Exemplo	Por que essa alternativa foi escolhida?	Benefícios, limitações ou compromissos envolvidos.

2.5 Restrições de arquitetura

Orientação: Registrem restrições que limitem as decisões de arquitetura ou implementação, como tecnologias obrigatórias, integrações com sistemas existentes, ambiente de execução, restrições organizacionais ou convenções. Incluam apenas restrições efetivamente relevantes ao projeto.

Capítulo 3

Projeto de Componentes

Orientação: Detalhem a organização interna das partes mais relevantes do sistema. O projeto de componentes pode ser representado graficamente, textualmente ou por tabelas, de acordo com a natureza da solução.

3.1 Visão geral dos componentes

Orientação: Listem os principais componentes e suas responsabilidades. Para sistemas maiores, esta visão geral pode cobrir todos os componentes, enquanto o detalhamento das seções seguintes pode se concentrar apenas nos mais relevantes ou complexos.

Tabela 3.1: Principais componentes do sistema.

Componente	Responsabilidade principal
Componente A	Descrever sua responsabilidade principal.
Componente B	Descrever sua responsabilidade principal.

3.2 Detalhamento dos componentes principais

Orientação: Seleccionem os componentes mais relevantes ou complexos e apresentem sua estrutura, responsabilidades, interfaces e dependências. Se o projeto for orientado a objetos, utilizem principalmente diagramas de classes UML e, quando necessário, complementem com diagramas de sequência, atividade ou outros artefatos.

3.3 Princípios de projeto

Orientação: Expliquem como os princípios de projeto discutidos na disciplina foram aplicados nos principais componentes ou classes do sistema. Por exemplo, podem ser discutidos aspectos relacionados à separação de responsabilidades, coesão, acoplamento, dependência de abstrações e princípios SOLID. Não apenas enumerem os princípios: indiquem onde foram aplicados e qual problema de projeto ajudaram a resolver. Para evidenciar sua aplicação, utilizem, quando adequado, diagramas de classes ou trechos de código-fonte.

3.4 Padrões de projeto

Orientação: Para cada padrão de projeto efetivamente utilizado, indiquem o problema que motivou sua aplicação, as classes ou componentes participantes, como o padrão foi aplicado e qual benefício se espera obter. Não utilizem padrões apenas para preencher esta seção.

Capítulo 4

Projeto de Interface do Usuário

Orientação: *Apresentem as interfaces e os principais fluxos de interação do usuário com o sistema. O principal artefato pode ser constituído por protótipos de baixa ou alta fidelidade. Diagramas de atividade, fluxos de navegação ou outros artefatos podem complementar a explicação.*

Não é necessário apresentar todas as telas. Priorizem as interfaces e os fluxos mais importantes para compreender o funcionamento do sistema.

Orientação: *Para cada fluxo principal, expliquem brevemente a tarefa do usuário e as decisões de interface relevantes. Se a equipe utilizar uma ferramenta externa, como Figma, apresentem exemplos representativos neste documento e forneçam o link para o material completo.*

Capítulo 5

Especificação e Estratégia de Testes

***Orientação:** Mostrem como a equipe pretende verificar o comportamento do software e reduzir o risco de defeitos. O capítulo deve apresentar uma estratégia coerente, e não apenas uma lista de ferramentas.*

5.1 Planejamento e níveis de teste

***Orientação:** Indiquem os níveis de teste aplicáveis ao projeto, por exemplo, unidade, integração e sistema, e expliquem o objetivo de cada um deles no contexto do software.*

5.2 Testes de unidade

***Orientação:** Seleccionem unidades relevantes e definam cenários e casos de teste. Quando existirem dependências externas ou difíceis de controlar, expliquem se serão utilizados stubs, mocks ou outras formas de substituição. Apresentem exemplos representativos contendo entradas, condições e resultados esperados.*

5.3 Testes de integração

***Orientação:** Expliquem como os componentes serão integrados e testados. Quando fizer sentido, indiquem a estratégia adotada (por exemplo, top-down, bottom-up, depth-first ou breadth-first) e justifiquem-na. Descrevam também como testes de regressão serão utilizados após alterações.*

5.4 Critérios de conclusão e cobertura

***Orientação:** Definam critérios que permitam decidir quando a atividade de teste atingiu um nível satisfatório. A cobertura de código pode ser um dos indicadores, mas não deve ser tratada isoladamente como garantia de qualidade. Se uma meta quantitativa for adotada, indiquem o tipo de cobertura e justifiquem a meta.*

5.5 Automação e ferramentas

***Orientação:** Descrevam as ferramentas utilizadas ou planejadas para executar testes, gerar relatórios e, quando aplicável, automatizar sua execução.*

Capítulo 6

Gestão de Configuração e Manutenção

***Orientação:** Este capítulo deve mostrar como os artefatos do software são versionados, modificados e mantidos de forma controlada. Relacionem as práticas descritas ao projeto da equipe, evitando apenas reproduzir definições teóricas.*

6.1 Repositório e itens de configuração

***Orientação:** Informem o repositório utilizado e identifiquem os principais itens de configuração que precisam ser controlados, por exemplo: código-fonte, arquivos de configuração, scripts, documentação, modelos de dados e testes.*

6.2 Gestão de dependências e alterações

***Orientação:** Expliquem como dependências externas são registradas e versionadas e como alterações relevantes são controladas. Quando uma alteração afetar vários artefatos, indiquem como a equipe mantém a coerência entre código, testes, documentação e demais elementos relacionados.*

6.3 Rastreabilidade

***Orientação:** Quando aplicável, mostrem como uma alteração pode ser rastreada entre os artefatos relacionados, por exemplo: necessidade/requisito, tarefa ou issue, componente afetado, commit e caso de teste. Não é necessário criar uma matriz extensa; um ou dois exemplos relevantes podem ser suficientes para demonstrar a prática.*

6.4 *Build*, integração e entrega

***Orientação:** Descrevam como o software é construído e executado. Se houver automação de build, testes, integração contínua ou entrega/deploy, apresentem o fluxo adotado e as ferramentas envolvidas. Caso essas práticas não sejam utilizadas, expliquem brevemente como o processo é realizado manualmente.*

Controle de Versões do Documento

Orientação: Registrem apenas alterações significativas nesta documentação. Este controle se refere ao documento; o versionamento do software deve ser descrito no capítulo de Gestão de Configuração e Manutenção.

Tabela 6.1: Histórico de versões do documento.

Data	Autor(es)	Versão	Alteração
DD/MM/AAAA	Equipe	1.0	Versão inicial para entrega.